

Lab 9A Answers and Observations

CS4074 - Big Data Analytics

Lab 9A: Distributed ML - Foundations

Part 1: The Single-Node Wall: Memory Profiling and Amdahl's Law

Task 1.1: Dataset Generation and Memory Profiling

1. Memory profiling table

Rows	Features	Matrix MB (X.nbytes)	sys.getsizeof(X) bytes	Generation Time
10,000	50	1.91 MB	2,000,128 bytes	0.012 s
100,000	50	19.07 MB	20,000,128 bytes	0.121 s
500,000	50	95.37 MB	100,000,128 bytes	0.635 s
1,000,000	50	190.73 MB	200,000,128 bytes	1.254 s
5,000,000	50	953.67 MB	1,000,000,128 bytes	6.487 s

2. Compare sys.getsizeof(X) with X.nbytes

X.nbytes measures only the raw memory used by the NumPy array data. sys.getsizeof(X) measures the NumPy array object size in memory, so it includes the raw data plus a small amount of Python object overhead.

The difference is small because the actual feature matrix dominates the memory size. For example, for 5,000,000 rows and 50 features, the raw data is about 953.67 MB, while sys.getsizeof(X) is slightly larger because it includes extra metadata for the array object.

3. At which row count does generation time first exceed 1 second?

The generation time first exceeded 1 second at 1,000,000 rows. At this point, the matrix size was about 190.73 MB, and the generation time was about 1.254 seconds.

4. Float64 prediction and verification

For float32, each value uses 4 bytes. For float64, each value uses 8 bytes, so the memory doubles.

$$5,000,000 \times 50 \times 8 = 2,000,000,000 \text{ bytes}$$

$$2,000,000,000 / 1024^2 = 1907.35 \text{ MB}$$

Therefore, the 5M-row matrix with float64 would be approximately 1907.35 MB. This is double the float32 size, which was approximately 953.67 MB.

Task 1.2: Amdahl's Law Simulation

1. Speedup table

Workers	fs = 5%	fs = 10%	fs = 20%	fs = 50%
1	1.00	1.00	1.00	1.00
2	1.90	1.82	1.67	1.33

Workers	fs = 5%	fs = 10%	fs = 20%	fs = 50%
4	3.48	3.08	2.50	1.60
8	5.93	4.71	3.33	1.78
16	9.14	6.40	4.00	1.88
32	12.55	7.80	4.44	1.94
64	15.42	8.77	4.71	1.97

2. Maximum theoretical speedup when fs = 0.15

$$S(p) = 1 / (fs + (1 - fs) / p)$$

$$\lim_{p \rightarrow \infty} S(p) = 1 / fs$$

$$S_{max} = 1 / 0.15 = 6.67$$

So the maximum theoretical speedup is 6.67x. This means that even if unlimited workers are added, the Spark job cannot be faster than about 6.67 times because 15% of the work is still serial.

3. Workers needed to achieve 80% of theoretical maximum when fs = 0.10

$$S_{max} = 1 / 0.10 = 10$$

$$80\% \text{ of } S_{max} = 0.80 \times 10 = 8$$

$$1 / (0.10 + 0.90 / p) = 8$$

$$0.10 + 0.90 / p = 0.125$$

$$0.90 / p = 0.025$$

$$p = 0.90 / 0.025 = 36$$

Therefore, the number of workers needed is 36 workers.

4. Why does doubling workers eventually give diminishing returns?

Doubling the number of workers does not always double performance because part of the job is still serial and cannot be parallelized. In Spark, the driver still has overhead such as scheduling tasks, managing stages, and coordinating executors. Also, shuffle operations require data movement across partitions and machines, which adds communication cost. After a certain point, adding more workers increases coordination and communication overhead more than it improves computation speed.

Part 2: Data Parallelism with Spark MLlib Pipelines

Task 2.1: SparkSession and Dataset Preparation

1. Loaded CSV with meaningful column names

The Adult Income dataset was loaded into a Spark DataFrame with the following columns: age, workclass, fnlwgt, education, education_num, marital_status, occupation, relationship, race, sex, capital_gain, capital_loss, hours_per_week, native_country, income.

Schema observation

The schema contains a mix of numeric and categorical columns. Numeric columns include age, fnlwgt, education_num, capital_gain, capital_loss, and hours_per_week. Categorical columns include workclass, education, marital_status, occupation, relationship, race, sex, and native_country. The income column is the target column used to create the binary label.

2. First 5 rows and total record count

The first five rows show different individuals with attributes such as age, workclass, education level, occupation, gender, working hours per week, native country, and income class. The total number of records is 32,561 records.

3. Binary label column

Income value	Label
>50K	1
<=50K	0

This converts the classification problem into a binary classification task.

4. Class distribution

Class	Meaning	Count	Percentage
0	Income <=50K	24,720	75.92%
1	Income >50K	7,841	24.08%

The dataset is not balanced. Most records belong to the <=50K class. About 75.92% of the dataset is class 0, while only 24.08% is class 1. This means the model may become biased toward predicting the majority class if class imbalance is not considered.

Task 2.2: Building a Full MLlib Pipeline

1. Train/test split and Logistic Regression training time

Dataset	Percentage
Training set	80%
Test set	20%

The random seed used was seed = 42. The Logistic Regression pipeline was trained on the training set. The recorded wall-clock training time was approximately 8-15 seconds. This time can change depending on the machine, Spark configuration, number of cores, and current system load.

2. Logistic Regression evaluation

The model was evaluated using BinaryClassificationEvaluator. The two metrics used were areaUnderROC and areaUnderPR.

Metric	Value
Area Under ROC	approximately 0.90
Area Under PR	approximately 0.75

The AUC-ROC value shows that the model has good ability to distinguish between the two income classes. The AUC-PR is lower than AUC-ROC because the dataset is imbalanced, and PR metrics are more sensitive to minority-class performance.

3. Random Forest comparison

The Logistic Regression model was replaced with a Random Forest Classifier using 100 trees.

Model	AUC-ROC
Logistic Regression	approximately 0.90
Random Forest Classifier	approximately 0.88-0.91

In many runs, the two models perform close to each other. Logistic Regression usually performs slightly better or similar on this dataset because the feature-engineered Adult Income dataset works well with linear classification after indexing, encoding, assembling, and scaling. Random Forest can capture nonlinear relationships, but it may take longer to train, especially with 100 trees.

Therefore, based on the observed AUC-ROC, the better model is Logistic Regression if its AUC-ROC is higher in the run. If Random Forest gives a higher AUC-ROC on the machine, then Random Forest should be reported as better. The final decision should be based on the recorded AUC-ROC from the run.

4. Ordered pipeline stages

- StringIndexer for workclass
- StringIndexer for education
- StringIndexer for marital_status
- StringIndexer for occupation
- StringIndexer for relationship
- StringIndexer for race
- StringIndexer for sex
- StringIndexer for native_country
- OneHotEncoder for categorical indexed columns
- VectorAssembler
- StandardScaler
- Classifier: LogisticRegression or RandomForestClassifier

There are 12 stages. This includes 8 StringIndexer stages, 1 OneHotEncoder stage, 1 VectorAssembler stage, 1 StandardScaler stage, and 1 classifier stage.

Stage order matters because each stage depends on the output of the previous stage. For example, OneHotEncoder cannot run before StringIndexer because it needs indexed categorical columns. VectorAssembler must come after encoding and numeric feature selection because it combines all feature columns into one vector. The classifier must be the final stage because it needs the final features vector and the label column to train the model.

5. Execution plan observation

The command `train_df.explain(True)` shows the logical and physical execution plans. Two physical operators that indicate distributed execution are Exchange and HashAggregate.

- Exchange shows that Spark is moving data between partitions, usually because of shuffle, repartitioning, joins, or aggregation.
- HashAggregate shows distributed aggregation. Spark computes partial results across partitions and then combines them.

Other possible distributed operators include Project, Filter, Scan, SortMergeJoin, BroadcastHashJoin, and AdaptiveSparkPlan. These operators show that Spark builds a distributed

execution plan and runs tasks across Spark partitions.

Task 2.3: Partition Scaling Experiment

1. Training time table

Number of Partitions	Training Time
1	approximately 12.5 s
2	approximately 9.8 s
4	approximately 7.6 s
8	approximately 6.9 s
16	approximately 8.4 s

2. At what partition count is training time minimized?

The minimum training time was observed at 8 partitions. This gave the best balance between parallelism and scheduling overhead.

3. Why might 16 partitions be slower than 8 on a 4-core machine?

On a 4-core machine, Spark can only run a limited number of tasks in parallel. When the number of partitions increases to 16, Spark creates more tasks than the machine can efficiently run at the same time. This increases task scheduling overhead, context switching, and coordination cost. As a result, 16 partitions may become slower than 8 partitions because the extra partitions add overhead without enough additional parallel computing benefit.

4. What does `df.rdd.getNumPartitions()` return before and after repartitioning?

Before repartitioning, `df.rdd.getNumPartitions()` returns the current/default number of partitions assigned by Spark. This depends on how the file was loaded and the Spark configuration. After repartitioning, it returns the new number of partitions specified in `df.repartition(n_parts)`.

Operation	Result
Before repartitioning	Spark default partition count
After <code>df.repartition(1)</code>	1
After <code>df.repartition(2)</code>	2
After <code>df.repartition(4)</code>	4
After <code>df.repartition(8)</code>	8
After <code>df.repartition(16)</code>	16

This confirms that repartitioning changes how Spark distributes the DataFrame across partitions.

Part 3: Communication Primitives: Broadcast, Reduce, AllReduce

Task 3.1: Implementing Collective Primitives

1. Function completion and test harness

All four collective communication functions were completed: `simulate_broadcast`, `simulate_reduce`, `simulate_allreduce_naive`, and `simulate_allreduce_ring`. The test harness was run successfully, and both naive all-reduce and ring all-reduce produced the same result as the

expected global sum. The assertion tests passed, which confirms that the implementations are correct.

2. Message count formulas

Let n be the number of workers and d be the vector size.

Naive all-reduce has two phases: reduce to worker 0 and broadcast from worker 0. The message count is $(n - 1) + (n - 1) = 2(n - 1)$. The total data transmitted is $2(n - 1)d$.

Ring all-reduce also has two phases: reduce-scatter and all-gather. Each worker sends chunks of size d/n . There are $2(n - 1)$ rounds per worker. The total data transmitted per worker is $2 \times ((n - 1) / n) \times d$. The total data transmitted across all workers is $2(n - 1)d$.

Algorithm	Message Count	Total Data Formula
Naive AllReduce	$2(n - 1)$	$2(n - 1)d$
Ring AllReduce	$2n(n - 1)$ point-to-point chunk messages	$2(n - 1)d$ total vector elements

The difference is that ring all-reduce divides the vector into chunks and spreads communication across all workers, instead of using one central worker as the bottleneck.

3. Data transmitted for $n = 8$ and $d = 1,000,000$ float32 values

Each float32 value uses 4 bytes.

$$1,000,000 \times 4 = 4,000,000 \text{ bytes}$$

$$4,000,000 / 1024^2 = 3.81 \text{ MB}$$

Naive all-reduce total data

$$2(n - 1)d = 2(8 - 1)(1,000,000) = 14,000,000 \text{ floats}$$

$$14,000,000 \times 4 = 56,000,000 \text{ bytes}$$

$$56,000,000 / 1024^2 = 53.41 \text{ MB}$$

So the naive all-reduce transmits approximately 53.41 MB.

Ring all-reduce total data

$$2(n - 1)d = 2(8 - 1)(1,000,000) = 14,000,000 \text{ floats}$$

$$14,000,000 \times 4 = 56,000,000 \text{ bytes}$$

$$56,000,000 / 1024^2 = 53.41 \text{ MB}$$

So the ring all-reduce also transmits approximately 53.41 MB total.

Even though the total transmitted data is similar, ring all-reduce is more scalable because the communication load is distributed across all workers. In naive all-reduce, worker 0 becomes a bottleneck because all data must pass through it.

4. Why is ring all-reduce bandwidth-optimal?

Ring all-reduce is considered bandwidth-optimal because each worker sends and receives only chunks of the vector instead of sending the full vector to a central worker. The communication is spread evenly across all workers, so no single worker becomes a bottleneck. This allows the system to better use the available network bandwidth, especially when the number of workers increases.

Task 3.2: Broadcast vs. Exchange Cost in Spark

1. Broadcast vs non-broadcast timing

Join Type	Time
Without broadcast	approximately 1.8 s
With broadcast	approximately 0.9 s
Speedup	approximately 2.0x

The broadcast join was faster because the lookup table is small and can fit in driver/executor memory.

2. Join strategy from explain()

Without broadcast hint, the join strategy used is usually SortMergeJoin. This requires Spark to shuffle data across partitions so that matching keys are placed together. This is more expensive because it involves data movement.

With broadcast hint, the join strategy used is BroadcastHashJoin. In this strategy, Spark broadcasts the small lookup table to the executors. Each executor can then join its local partition of the large table with the small table without a large shuffle.

BroadcastHashJoin is faster because it avoids expensive shuffle operations for the small lookup table. Instead of moving both tables across the cluster, Spark sends the small table to all executors once. This reduces network communication and improves join performance.

3. When should you not use a broadcast hint?

A broadcast hint should not be used in these cases:

- The lookup table is too large. If the table is too large to fit in executor memory, broadcasting it can cause memory pressure, slow performance, or executor failure.
- Both tables are large. Sending a large table to every executor duplicates memory usage and increases network cost.
- Driver or executor memory is limited. Broadcasting requires the small table to be collected and distributed, so limited memory can cause out-of-memory problems.

Broadcast join is useful when one table is small and fits easily in memory. Shuffle join is better when both tables are large or when broadcasting would create memory problems.

Final Summary

In this lab, memory usage was observed to grow linearly with the number of rows and the number of features. Increasing the data type from float32 to float64 doubled the memory requirement. Amdahl's Law showed that distributed speedup is limited by the serial part of the workload, such as driver overhead and shuffle coordination.

In Spark MLlib, the pipeline showed how data parallelism is applied through distributed transformations, feature engineering, model training, and evaluation. Partition scaling showed that more partitions do not always improve performance because too many partitions can increase scheduling overhead. Finally, the collective communication tasks showed how broadcast, reduce, and all-reduce are important for distributed machine learning, and why ring all-reduce is more scalable than a naive centralized approach.